

Project Approach

This document details the architectural decisions, structural patterns, and implementation strategy used to build the Shoe Cart React Native application.

1. Routing & Dual-Portal Approach (Expo Router)

User roles are separated at the routing level using Expo Router's file-based directory groups:

- **Role Selection (`app/index.tsx`):** The entry point that lets users switch between customer shopping and admin store management.
 - **Customer Portal (`app/(customer)/`):** Contains the main shopping experience under a tab layout (`shop.tsx`, `cart.tsx`, `orders.tsx`, `profile.tsx`), with the bottom navigation bar always visible.
 - **Admin Portal (`app/(admin)/`):** Contains store management screens (`orders.tsx`, `inventory.tsx`, `profile.tsx`, and `manage.tsx`), letting admins track inventory and update order statuses.
 - **Global Modals (`app/checkout.tsx`):** Placed at the root level so checkout appears as a full-screen view on top of everything else, hiding the bottom tab bar for a more focused flow.
-

2. State Management Approach (Redux Toolkit)

To keep everything working locally without needing a backend server, Redux Toolkit was used across three main slices:

- **`productSlice.ts`:** Manages the product catalog, letting admins add or edit items in real time.
 - **`cartSlice.ts`:** Handles item selection, size, quantity, and removal.
 - **`orderSlice.ts`:** Saves completed orders to order history and lets admins update order status (Processing → Shipped → Delivered).
-

3. Separation of Concerns (Custom Hooks Pattern)

To keep components clean, UI screens are kept strictly presentation-only. All business logic, calculations (like subtotal and delivery fees), search filtering, and Redux dispatches were moved into custom hooks in `src/hooks/`:

- `useProductSearch.ts` — filters products by name and brand.
 - `useCart.ts` — computes totals and manages cart actions.
 - `useAdminOrders.ts` — handles order status updates.
 - `useManageProduct.ts` — manages form state for adding/editing products.
 - `useUserAccount.ts` / `useAdminProfile.ts` — handles navigation and role switching.
-

4. Form Validation & Data Integrity

To keep the app type-safe and prevent bad data:

- **Strict TypeScript:** No `any` types are used anywhere in the codebase.
 - **Zod + React Hook Form:** Admin product forms and the checkout form are backed by Zod schemas, which validate pricing, card numbers, expiry dates, CVVs, and sizes before anything is submitted.
-

Constraints & What Was Omitted

Unit Testing: Due to a tight deadline (July 31, 2026), automated unit and integration tests (Jest, React Native Testing Library) weren't included. Time was spent instead on the UI, routing, type safety, and state management, since those mattered most for this version.

Future Improvements & Roadmap

If given more time, these would be the top priorities:

1. **Automated Testing Suite** — Add Jest tests for Redux slices and custom hooks, plus React Native Testing Library for component tests.
2. **Backend Database Integration** — Replace local state with Supabase or Firebase for cloud-synced inventory and multi-user authentication.

3. **Real Payment Gateway Integration** — Swap the simulated checkout for a real Stripe or Apple Pay integration.
4. **Dedicated Wishlist Screen** — Expand the favorite toggle into a proper wishlist tab tied to the user's profile.
5. **Admin Analytics Dashboard** — Add charts for revenue, top-selling brands, and low-stock alerts.